Sr Platform Engineer Technical Assessment

- Overview
  - AI Tooling Policy
  - Environment Context
  - Scoring Rubric
  - What We Look For
- Your Submission/Responses
  - Section 1: Code Review (~15 min)
  - Section 2: Design Question (~15 min)
  - Section 3: Service Deployment Rationale (~5 min of your 30)
    - Example: notification-sender-dev

Overview

This assessment is designed to take approximately **1 hour**. Your submission will be used as a basis for technical interviews - we'll ask you to walk through your decisions and discuss alternatives.

Section	Format	Time
1. Code Review	Written PR feedback	~15 min
2. Design Question	Written response	~15 min

3. Service Deployment	Terraform code	~30 min

AI Tooling Policy

We expect you to use AI tools (Copilot, Claude, Codex, etc.) as you would in your daily work. We're interested in **how** and **why** you design/decide not just **what** you can output.

We ARE testing:

- Your judgment about security, reliability, and trade-offs
- Your ability to evaluate and validate output - whether from AI or a teammate
- Your understanding of **why** infrastructure is designed a certain way

Note which AI tools you used in your submission. This is a positive signal, not a penalty.

Environment Context

You're joining a team that runs:

- **AWS**: Multi-account (`dev` / `uat` / `prod`)
- **EKS**: Kubernetes, Istio service mesh, hosting for Ruby on Rails
- **Databases**: Per-service RDS PostgreSQL + ElastiCache Redis
- **IaC**: Terraform Cloud
- **Pod Identity**: IRSA (IAM Roles for Service Accounts) scoped per service
- **State Sharing**: `terraform_remote_state` between workspaces hosted in Terraform Cloud

Scoring Rubric

We share this so you know what we value. Your submission will be used as a starting point for technical interviews.

Criteria	Weight	What "Strong" Looks Like
Code review depth (Section 1)	30%	Catches critical security issues, provides actionable suggestions with "why", prioritizes appropriately

Design reasoning (Section 2)	25%	Concrete incident scenario, pragmatic trade-offs, not just textbook least-privilege
Service deployment (Section 3)	30%	Production config reflects payment workload needs (HA, sizing, backups), clear dev/prod differentiation driven by business context
Communication clarity	15%	Concise, specific, grounded in experience rather than generic

What We Look For

Strong signals:

- Code review catches security issues AND explains the blast radius
- Suggestions are specific (e.g., "use Secrets Store CSI" not just "don't hardcode secrets")
- Design answer includes a realistic incident scenario, not just theory
- Prod config shows awareness of payment workload characteristics (peak dates, financial data sensitivity, HA requirements)
- Dev/prod differences are intentional and explainable, not just "bigger numbers"
- Honest about what they don't know or would add with more time

Concerns:

- Hardcoded credentials not flagged in code review
- Wildcard IAM permissions not called out
- Design answer that's purely theoretical with no concrete scenarios
- Prod config that's just dev with bigger instance types and no reasoning
- Missing multi-AZ, backup retention, or skip_final_snapshot=false for a financial service in prod

Your Submission/Responses

Submit your answers for the following 3 sections using the supplied `RESPONSES.md` file. Please push your completed `RESPONSES.md` as well as any supplemental code to a new Github repository and provide the repository URL within ~24hours of receiving this challenge for us to review.

Section 1: Code Review (~15 min)

Question: A teammate has submitted the below HCL file as a PR to deploy a new Rails service to EKS.

Answer: Write your feedback as you would in a real GitHub PR review. For any issue state **what's wrong**, **why it matters**, and **what you'd suggest instead**. Type your responses into into the supplied `RESPONSES.md` file.

review/pull_request.tf:

```
# ==========================================================================
# Section 1: Code Review
# ==========================================================================
#
# A teammate has submitted this Terraform code as part of a PR to deploy a new
# Rails service called "my-service" to the EKS cluster. The service needs:
#   - An IAM role for pod-level AWS access (IRSA)
#   - A Kubernetes deployment running the Rails app
#
# Review this code and provide feedback as you would in a real
# GitHub PR review. For each issue: what's wrong, why it matters, and what
# you'd suggest instead.
# ==========================================================================

resource "aws_iam_role" "service_role" {
  name = "eks-service-role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect    = "Allow"
      Principal = { Federated = "arn:aws:iam::123456789012:oidc-provider/oidc.eks.us-east-2.amazonaws.com/id/EXAMPLE" }
      Action    = "sts:AssumeRoleWithWebIdentity"
    }]
  })
}
```

```
27
28 resource "aws_iam_role_policy" "service_policy" {
29   name = "service-access"
30   role = aws_iam_role.service_role.id
31
32   policy = jsonencode({
33     Version = "2012-10-17"
34     Statement = [
35       {
36         Effect   = "Allow"
37         Action   = ["ssm:*", "secretsmanager:*", "kms:*"]
38         Resource = "*"
39       },
40       {
41         Effect   = "Allow"
42         Action   = ["s3:*"]
43         Resource = "*"
44       }
45     ]
46   })
47 }
48
49 resource "kubernetes_deployment" "app" {
50   metadata {
51     name      = "my-service"
52     namespace = "default"
53   }
54
55   spec {
56     replicas = 3
57
58     service_account_name = aws_iam_role.service_role.name
59
60     selector {
61       match_labels = {
62         app = "my-service"
63       }
64     }
65
66     template {
67       metadata {
68         labels = {
69           app = "my-service"
70         }
71       }
72
73       spec {
74         container {
75           name  = "app"
76           image = "123456789012.dkr.ecr.us-east-2.amazonaws.com/my-
service:latest"
77
78           port {
79             container_port = 3000
80           }
81
82           env {
83             name  = "DATABASE_URL"
84             value = "postgresql://admin:password123@mydb.cluster-
xyz.us-east-2.rds.amazonaws.com:5432/myservice"
85           }
86
87           env {
88             name  = "REDIS_URL"
89             value = "redis://my-
redis.abc123.ng.0001.use2.cache.amazonaws.com:6379"
90           }
```

```
        }
      }
    }
  }
}
```

Section 2: Design Question (~15 min)

Question: Each Rails service gets its own IAM role via IRSA (IAM Roles for Service Accounts), scoped to a service-specific secrets path:

```
arn:aws:ssm:{region}:{account}:parameter/{service-name}/*
```

A developer asks: *"Can we just create one shared IAM role for all services? They all need the same permissions ie SSM, Secrets Manager, KMS decrypt, and EventBridge PutEvents. Separate roles are a maintenance burden!"*

Answer: Write your feedback as you would give to the team, type your responses into into the supplied `RESPONSES.md` file making sure to address:

- Why you would or would not adopt a shared role
- If you reject it, propose a **practical alternative** that reduces overhead without removing the security boundary
- Describe a **concrete scenario** where per-service roles prevent a real incident For any issue state **what's wrong**, **why it matters**, and **what you'd suggest instead**.

Section 3: Service Deployment Rationale (~5 min of your 30)

A new backend service called "payment-processor" is ready to deploy. It handles loan payment processing and disbursement.

<u>Terraform context:</u>
Our team makes use of a common terraform module called `service-stack` that deploys a backend service. The common `service-stack` terraform module provisions:

- **PostgreSQL database** (RDS)
- **Redis cache** (ElastiCache)
- **Application containers** (Kubernetes Deployment)
- **Background workers** (optional Kubernetes Deployment)

The module is used identically across `dev` and `prod` with any and all environment differences are expressed through variable values.

Module Variables

```
# --- Required ---
variable "service_name"       {}                # string — used for naming all resources
variable "vpc_id"             {}                # string — from network workspace
variable "db_subnet_group"    {}                # string — from network workspace
variable "cluster_name"       {}                # string — EKS cluster name from network workspace
variable "namespace"          { default = "services" }

# --- Database (RDS PostgreSQL) ---
variable "db_instance_class"         { default = "db.t3.micro" }
variable "db_storage_gb"             { default = 20 }
variable "db_max_storage_gb"         { default = 100 }    # autoscaling ceiling
variable "db_multi_az"               { default = false }
variable "db_backup_days"            { default = 7 }       # automated backup retention
variable "db_skip_final_snapshot"    { default = true }
variable "db_encryption"             { default = true }
variable "db_performance_insights"   { default = false }

# --- Cache (ElastiCache Redis) ---
variable "cache_node_type"           { default = "cache.t3.micro" }
variable "cache_cluster_mode"        { default = false }   # true = multi-node with failover
variable "cache_snapshot_days"       { default = 0 }       # snapshot retention (0 = disabled)

# --- Application Containers ---
variable "app_replicas"              { default = 1 }
variable "app_cpu_request"           { default = "250m" }
variable "app_memory_request"        { default = "512Mi" }
variable "app_cpu_limit"             { default = "500m" }
variable "app_memory_limit"          { default = "1Gi" }

# --- Background Workers (optional) ---
variable "enable_workers"            { default = false }
variable "worker_replicas"           { default = 1 }
variable "worker_cpu_request"        { default = "250m" }
variable "worker_memory_request"     { default = "512Mi" }
variable "worker_cpu_limit"          { default = "500m" }
variable "worker_memory_limit"       { default = "1Gi" }
```

Example: `notification-sender-dev`

An existing lightweight service that sends emails and push notifications in the `dev` environment:

```
module "notification_sender" {
  source = "../modules/service-stack"

  service_name    = "notification-sender"
  vpc_id          = data.terraform_remote_state.network.outputs.vpc_id
  db_subnet_group = data.terraform_remote_state.network.outputs.db_subnet_group
```

```
  cluster_name    = data.terraform_remote_state.network.outputs.cluster_name

  # Database — small, low-traffic
  db_instance_class = "db.t3.micro"
  db_storage_gb     = 20

  # Cache — single node
  cache_node_type = "cache.t3.micro"

  # App — 1 replica
  app_replicas = 1

  # Workers — sends emails async
  enable_workers  = true
  worker_replicas = 1
}
```

"payment-processor" context:

- this new service will process financial transactions and stores payment records
- this new service will receive payment events asynchronously via background workers
- this new service will see peak traffic on the 1st and 15th of each month (payment due dates)

Question: You are tasked with writing/adding the needed `service-stack` module use/calls for this newly added **payment-processor** service in both the `dev` and `prod` environments.

Answer: Provide your HCL submission typed into into the supplied `RESPONSES.md` file making sure to address:

- **Why** you chose specific values for a payment workload
- **What** differs between **dev** and **prod** and **why**
- **What** you'd add with more time